

AaraMinds Platform — Project Overview

Executive summary. This repository is the canonical AaraMinds workspace and the implementation home for the AaraMinds Agent Platform (AAP). It combines a proof-grade local AAP runtime harness, schemas and contracts, the BA Agent reference package, engineering skills and MCP tooling, and the Instruction OS persona layer. The most important boundary: **platform/** is a deterministic proof harness for validating AAP contracts before hosted runtime binding; it is **not** the production runtime.

Metadata and evidence boundary

Field	Value
Repository	/Users/rb692q/projects/aaraminds-platform
Evidence commit	a926c5abb45c3024f0f30a0ae5ff4a931d2202e8
Document date	2026-07-03
Document purpose	Repository orientation for senior engineers, architects, and future AI coding sessions
Production readiness scope	Included as a boundary assessment only; this is not a production-readiness certificate
Evidence rule	Claims are grounded in repository code, config, tests, schemas, CI, generated proof output, or explicit status docs at the commit above

What this project is and is not

It is	Evidence anchors	Engineering meaning
The canonical AaraMinds workspace and AAP implementation home	README.md, .claude/CLAUDE.md	Work in this repo wins over historical snapshots or copied workspaces.
A file-defined AAP v1 proof workspace	governance/PRD_AaraMinds_Agent_Platform_v1.3.md, docs/runtime-verification-notes.md	AAP is modeled through manifests, schemas, tool contracts, proof flow, release gates, and guardrails.

It is	Evidence anchors	Engineering meaning
A deterministic local runtime proof harness	<code>platform/</code> , <code>platform/cmd/aapctl/main.go</code> , <code>platform/internal/runtime/*</code>	Engineers can validate manifest loading, tool-boundary decisions, audit shapes, trace-shaped records, and memory isolation without a hosted agent runtime.
A reference BA Agent package	<code>examples/ba-agent.manifest.yaml</code> , <code>skills-pack/agent-packages/aara-business-analyst/</code>	The BA Agent is the reference proof vehicle for manifest-controlled, evidence-backed, human-gated agent behavior.
A skills, agents, and deterministic MCP tooling workspace	<code>skills-pack/</code> , <code>skills-pack/.claude/INDEX.md</code> , <code>skills-pack/examples/microservices-system-design-mcp-server/</code>	Engineering knowledge is represented as Claude Skills, Claude/Copilot agents, and a Go MCP server with deterministic architecture tools.
A communication/persona instruction system	<code>instruction-os/README.md</code> , <code>instruction-os/Persona/README.md</code>	Communication personas and modules are maintained separately from engineering skills and runtime contracts.

It is not	Evidence anchors	Boundary
Not the production runtime	docs/runtime-verification-notes.md	The local harness proves contract semantics. Hosted runtime binding, identity, live telemetry, managed deployment, and production memory remain open or deferred.
Not a generic external agent-builder product in v1	governance/PRD_AaraMinds_Agent_Platform_v1.3.md	AAP v1 is internal-first and disciplined around AaraMinds agents.
Not proof that hosted Claude Agent SDK / Foundry runtime assumptions hold	docs/runtime-verification-notes.md , PRD runtime section	Runtime SDK, managed target, OTel GenAI, identity, and memory choices are current-edge assumptions marked [VERIFY] in repo docs.
Not a production-hardened MCP architecture-review service	skills-pack/examples/microservices-system-design-mcp-server/README.md	The example server is deterministic and tested, but production requires authn/authz, rate limits, audit forwarding, redaction, secret management, CORS, and deployment hardening.
Not a substitute for human approval or architecture judgement	docs/release-gate-thresholds.md , BA Agent package docs, demo README	Drafting, review, and guardrail automation exist; final approval, business commitments, production-impacting actions, and architecture acceptance remain human-owned.

Stable core, current edge

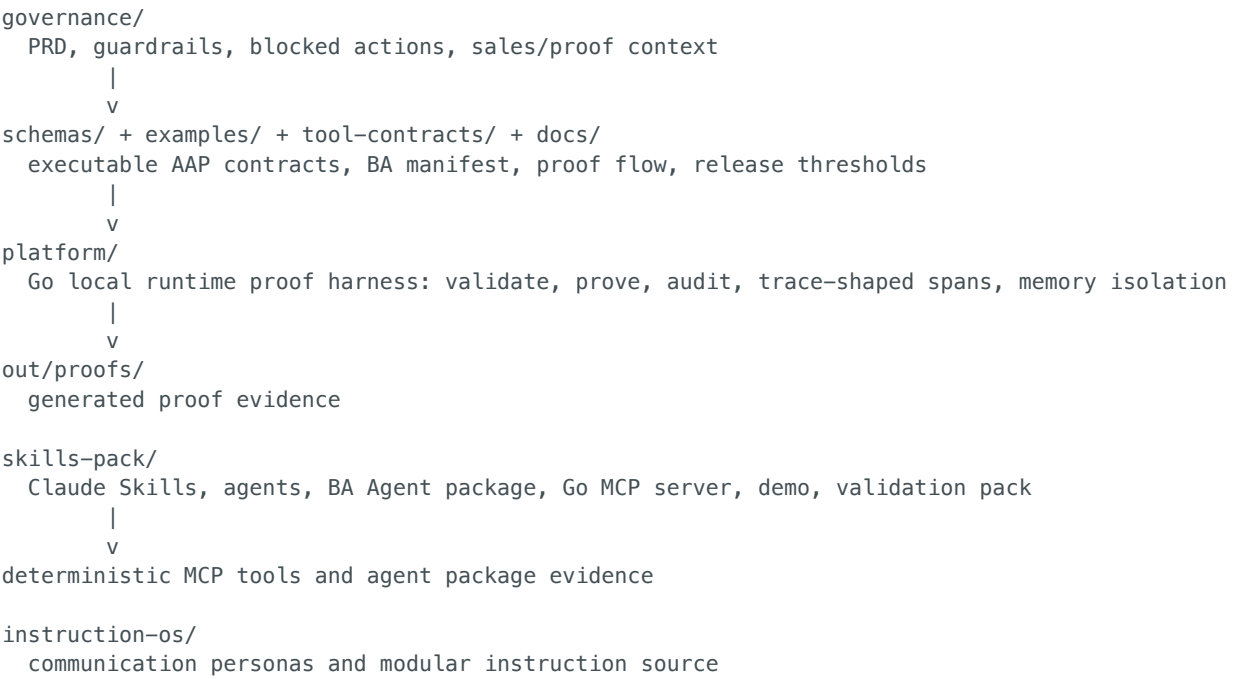
The durable architecture principles are independent of vendor runtime details:

Stable core principle	Where it appears	Why it matters
Manifest-controlled execution	<code>schemas/agent-manifest.schema.json</code> , <code>examples/ba-agent.manifest.yaml</code> , <code>platform/internal/runtime/engine.go</code>	The runtime should not start or call tools outside the declared manifest.
Contract-pinned tool calls	<code>schemas/mcp-tool-contract.schema.json</code> , <code>tool-contracts/*.contract.yaml</code>	Tool inputs, output expectations, permissions, failure modes, audit payload shape, and version pins are explicit.
Default-deny and human-gated high-risk actions	<code>governance/aap-blocked-actions.yaml</code> , <code>governance/aap-guardrails-checklist.md</code> , runtime tests	Off-manifest, missing-contract, blocked-action, and unattended soft-approval paths fail closed or escalate.
Scoped memory with citations	<code>schemas/memory-record.schema.json</code> , <code>platform/internal/runtime/memory.go</code> , runtime tests	Memory writes require classification and source citation; reads are scoped to the active engagement/agent policy.
Auditability and replayable evidence	<code>schemas/audit-event.schema.json</code> , <code>out/proofs/phase1-proof.json</code>	Governed actions produce audit events and proof reports.
Deterministic validation before model/runtime binding	<code>docs/runtime-verification-notes.md</code> , <code>skills-pack/demo/architecture-review-demo/</code>	Determinism keeps core platform semantics testable without relying on LLM variability or hosted-runtime maturity.

Current-edge items require re-verification before production commitment:

- Claude Agent SDK runtime extension points and deployment fit: `[VERIFY]`.
- Foundry Agent Service managed target, Entra Agent ID mapping, BYO VNet/private MCP subnet, and secrets model: `[VERIFY]`.
- OTel GenAI semantic convention maturity and Grafana compatibility: `[VERIFY]`.
- Azure managed identity per `agent_id` and local-dev fallback: `[VERIFY]`.
- Mem0 OSS + Azure OpenAI memory extraction quality: `[VERIFY]`.
- MCP SDK/spec compatibility and hosting guidance beyond the pinned repository versions: `[VERIFY]`.

System map and layered architecture



Layer	Primary paths	What is implemented	Operational boundary
Repository source of truth	README.md , .claude/CLAUDE.md , Ranking.md	Canonical workspace rules, inventory/ranking context, AI-session behavior guidance	Historical snapshots should not override this repo. Some prose inventory can lag generated indexes; verify with filesystem/coo
AAP governance	governance/PRD_AaraMinds_Agent_Platform_v1.3.md , governance/aap-guardrails-checklist.md , governance/aap-blocked-actions.yaml	AAP v1 proof goals, guardrails, blocked action taxonomy	PRD intent is n equivalent to implementation runtime code/tests decide implemented behavior.
AAP contract layer	schemas/ , examples/ba-agent.manifest.yaml , tool-contracts/	JSON schemas, BA manifest, pinned tool contracts	Schema/contr changes can invalidate runtime loading proof reports, and release gates.

Layer	Primary paths	What is implemented	Operational boundary
AAP proof runtime	<code>platform/</code>	Go module pinned to <code>go 1.25.5</code> ; <code>aapctl validate</code> and <code>aapctl prove</code> ; runtime engine, loader, memory store, schema validation, tests	Proof-grade lo harness only; hosted identity live OTEL exporter, persistent memory backend, or production deployment.
Proof evidence	<code>out/proofs/phase1-proof.json</code>	Generated report with manifest start, allowed tool execution, off-manifest denial, approval escalation, memory isolation, audit events, trace count	Running <code>aapctl prove</code> rewrite this artifact; tre diffs as evidence changes, not scratch output
Engineering knowledge and agents	<code>skills-pack/.claude/skills/</code> , <code>skills-pack/.claude/agents/</code> , <code>skills-pack/.claude/INDEX.md</code>	Generated index lists 35 skills, 17 agents, 3 hooks at 2026-06-18	Skills are mostly structured knowledge; do not infer behavioral validation unless eval outputs or executable tests exist.
MCP architecture tools	<code>skills-pack/examples/microservices-system-design-mcp-server/</code>	Deterministic Go MCP server with 13 tools, service/tool layering, contracts, tests, Dockerfile	Example/startup grade; production needs authz, rate limiting, audit sink, redaction, secret management, CORS, and deployment operations.
MCP demo validation	<code>skills-pack/demo/architecture-review-demo/</code>	Python stdlib MCP client calls five tools across three architectures and validates output against goldens	Validates deterministic drift, not production service fitness human

Layer	Primary paths	What is implemented	Operational boundary
			architecture correctness.
Instruction OS	<code>instruction-os/Persona/</code> , <code>instruction-os/skills/</code>	Active persona source, modular load order, role-based communication personas, validation artifacts	Edit Persona as source; dat references age and should no be treated as current ecosystem evidence.
Diagrams	<code>diagrams/</code> , <code>Repo_Context_Platform_Architecture*.md</code> , <code>*.mermaid</code>	Architecture/product visuals	Useful for orientation; implementation claims still need code/config/te evidence.

AAP proof harness

Lifecycle

The proof harness lifecycle is intentionally small and deterministic:

- 1. Load schemas and structured files.** **NewEngine** loads runtime schemas, validates the manifest against `schemas/agent-manifest.schema.json`, loads tool contracts from `tool-contracts/`, validates each contract against `schemas/mcp-tool-contract.schema.json`, validates each `example_invocation` against its tool `input_schema`, and loads blocked actions from the manifest reference.
- 2. Validate manifest semantics.** The engine checks required agent identity fields, supported runtime/status, payload-mode rules for active/platform-ready manifests, approval default rules, memory scope, blocked-action defaults, allowed skill source paths, allowed tool contract pins, boundary parity between manifest and contract, and evaluation gate references.
- 3. Start a run.** **Start** requires `run_id`, `engagement_id`, `user_id`, and `tenant_namespace`; defaults agent and manifest version from the manifest when absent; rejects mismatches; records a trace-shaped `agent.run` span plus `agent_started` and `manifest_validated` audit events.
- 4. Process tool requests through the gateway.** **InvokeTool** enforces run-start, manifest allowlist, contract presence, contract version, blocked action types, contract input schema, engagement scope, blocked boundaries, hard approvals, and unattended soft-approval escalation. It records audit events and trace-shaped spans for success, denial, and approval-required outcomes.
- 5. Write and query memory.** **WriteMemory** requires an active run, enabled manifest memory, matching agent and engagement, source citation matching the active run, schema-valid memory records, allowed classification, and PII policy. **QueryMemory** returns only records in the active engagement and allowed scope.

6. **Produce proof evidence.** `RunPhase1Proof` executes a fixed scenario and `aapctl prove` writes `out/proofs/phase1-proof.json`.

Contract and enforcement matrix

Concern	Source of truth	Runtime enforcement
Agent manifest shape	<code>schemas/agent-manifest.schema.json</code>	<code>ValidateStructuredFile</code> during engine construction
Manifest semantic rules	<code>platform/internal/runtime/engine.go</code>	Runtime/status, payload mode, approval defaults, memory scope, skill paths, evaluation refs
Tool contract shape	<code>schemas/mcp-tool-contract.schema.json</code>	<code>LoadContractsWithSchema</code> validates files and example invocations
Tool input schema	Each <code>tool-contracts/*.contract.yaml</code>	<code>InvokeTool</code> validates payload before execution/approval
Tool allowlist/version pins	<code>examples/ba-agent.manifest.yaml</code>	<code>InvokeTool</code> denies off-manifest, missing-contract, and version-mismatch calls
Blocked actions	<code>governance/aap-blocked-actions.yaml</code>	<code>InvokeTool</code> denies contract action types such as <code>production_delete</code>
Approval boundaries	Manifest plus tool contract	Soft boundaries require confirmation; unattended soft becomes hard; hard blocks execution until approval
Audit event shape	<code>schemas/audit-event.schema.json</code> plus per-contract <code>audit_event_schema</code>	Runtime validates run-scoped audit events and tool-specific audit payloads
Memory shape and scope	<code>schemas/memory-record.schema.json</code> , manifest memory policy	Runtime validates classification, citation, PII policy, agent/engagement match, and scoped query

Evidence produced by the current proof

`out/proofs/phase1-proof.json` records:

Proof field	Current value	Meaning
<code>valid_manifest_started</code>	<code>true</code>	BA manifest can be loaded and started by the local harness.

Proof field	Current value	Meaning
<code>allowed_tool_executed</code>	<code>true</code>	<code>get_project_context</code> succeeds when manifest, contract, version, schema, and scope align.
<code>off_manifest_tool_denied</code>	<code>true</code>	<code>delete_production_record</code> is denied when not in the manifest.
<code>denial_audit_logged</code>	<code>true</code>	The denial has a matching <code>tool_denied</code> audit event.
<code>soft_unattended_escalated</code>	<code>true</code>	<code>create_requirements_draft</code> soft approval becomes hard in unattended mode.
<code>memory_leakage_returned</code>	<code>0</code>	Querying another engagement returns no memory records.
<code>run_scoped_audits_have_run_id</code>	<code>true</code>	Run-scoped audit events carry <code>run_id</code> .
<code>trace_span_count</code>	<code>4</code>	The harness emits trace-shaped records for the proof scenario.

Coverage boundary

`docs/ba-agent-proof-flow.md` defines a 13-step BA Agent proof flow. The current implementation proves steps **2 through 10**: manifest load/validation, run initialization, audit start, scoped memory access, tool gateway checks, execution/denial/approval decisions, and tool/audit recording.

It does **not** prove steps 11 through 13 in the local harness: BA output quality separation, package evaluation, or converting a real production failure into a `SkillRevision` with before/after benchmark evidence. Those remain release/evaluation-layer concerns, not local runtime proof.

Skills-pack and MCP server

The skills-pack turns engineering knowledge into three runnable or semi-runnable forms:

Form	Primary paths	Evidence-backed claim	Boundary
Claude Skills	<code>skills-pack/.claude/skills/</code> , <code>skills-pack/.claude/INDEX.md</code>	Generated index lists 35 Tier-1 skills with reference counts and last-updated dates.	Skills route and structure expert guidance; most are not behaviorally validated unless backed by

Form	Primary paths	Evidence-backed claim	Boundary
			eval outputs.
Agents	<code>skills-pack/.claude/agents/</code> , <code>skills-pack/agent-packages/</code>	Generated index lists 17 agents; BA Agent has package artifacts, eval results, monitoring spec, rollback runbook, and adapter contracts.	Agent behavior needs live-session or explicit eval evidence. Do not infer production readiness from a design score alone.
Go MCP server	<code>skills-pack/examples/microservices-system-design-mcp-server/</code>	Server README and <code>internal/mcpserver/server.go</code> show 13 deterministic architecture tools; CI builds and tests it.	Not production-hardened; deterministic rule outputs are design-time guardrails, not human architecture acceptance.
Architecture-review demo	<code>skills-pack/demo/architecture-review-demo/</code>	Python runner performs real MCP stdio handshake and calls five tools across three architecture fixtures; validator compares JSON outputs to goldens.	Golden matching detects deterministic drift, not semantic superiority or production fitness.

MCP server structure

The example MCP server is organized to keep protocol wiring separate from rule logic:

Path	Responsibility
<code>cmd/server/main.go</code>	Parses transport environment, configures JSON <code>slog</code> to stderr, constructs the server, starts stdio/streamable HTTP/SSE transport.

Path	Responsibility
<code>internal/mcpserver/server.go</code>	Composition root; constructs the MCP server and registers tools.
<code>internal/services/<tool>/</code>	Deterministic rule logic, one service package per capability.
<code>internal/tools/<tool>/register.go</code>	MCP request parsing, validation, service invocation, JSON marshalling, structured tool-call logs.
<code>contracts/architecture-tools/implemented/</code>	Tool contracts for implemented architecture tools.
<code>testdata/</code>	Input fixtures and expected outputs for service-level tests.

The design trade-off is deliberate: deterministic services are reproducible and golden-testable, but they are bounded by encoded rules. They help a coding session avoid obvious architecture misses; they do not replace architect judgement or environment-specific review.

BA Agent package evidence

The BA Agent package is stronger than a paper design but still not a live production deployment:

- `eval-results.json` records 6/6 golden cases passed across 3 trials with `pass_at_k = 1.0` and `pass_caret_k = 1.0`.
- `release-gate.json` marks `decision: PASS` for a production-candidate stage while keeping live monitoring, rollback kill switch, and canary evidence as not yet complete.
- `monitoring-spec.md`, `rollback-runbook.md`, and `mcp-adapter-contracts.md` define production-stage controls, but the package states those controls must be stood up and exercised in the production runtime before production.

Known doc drift: `eval-plan.md` still says "Designed, not executed" while `eval-results.json`, `review-scorecard.md`, and `release-gate.json` say execution happened on 2026-06-18. For behavior claims, use `eval-results.json` as the higher-specificity evidence and update stale prose when touching the package.

Instruction OS

`instruction-os/` is the communication/persona layer, not the AAP runtime. Its operating model is:

- Active source lives under `instruction-os/Persona/`.
- Load `01_Layered_Base_System_v1.1.md`.
- Add only the relevant task module.
- Add a role-based persona when needed.
- Treat dated references and validation artifacts as support evidence, not active source.

Role-based personas are compositions, not replacements for the base system. The active persona list includes content strategy, AI agent blueprinting, AI engineering architecture, business strategy, executive narrative, and project planning. Future AI coding sessions should use this layer for communication shape and audience fit, while using `skills-pack/` and `platform/` for engineering and runtime claims.

Validation model

CI gates

`.github/workflows/ci.yml` defines the repository-level verification path:

Area	CI action	What it proves	What it does not prove
Platform formatting	<code>gofmt -l ./platform</code>	Go formatting in the AAP harness is clean.	No semantic correctness beyond formatting.
Platform tests	<code>go test ./..., go test -race ./..., go vet ./... in platform/</code>	Runtime unit tests, race checks, and vet checks pass. Tests cover manifest start, missing manifest, off-manifest denial, audit run IDs, approval escalation, telemetry payload rejection, YAML/contract schema validation, pre-start denial, input schema violation, engagement mismatch, blocked actions, memory isolation, citation run matching, and phase-1 proof. Runtime code also implements missing-contract and contract-version denial paths; direct test coverage for those paths should be verified before relying on them.	No hosted runtime, live identity, persistent database, real OTel exporter, or production memory backend.
Platform validate command	<code>go run ./cmd/aapctl validate</code>	BA manifest and tool contracts load and validate under the harness.	Does not write proof output or prove every release-gate threshold.
MCP formatting	<code>gofmt -l</code> over MCP server <code>cmd</code> and <code>internal</code>	MCP server Go formatting is clean.	No contract/semantic proof.
MCP tests/build	<code>go test ./..., go test -race ./..., go vet ./..., go build -buildvcs=false -o ./mcp-server ./cmd/server</code>	MCP services and tool wiring compile, test, race-check, vet, and build.	No production authn/authz, load test, or hosted deployment validation.
Demo goldens	<code>make demo, make validate in skills-</code>	A real stdio MCP client can call five tools across three fixtures	Only validates deterministic drift for

Area	CI action	What it proves	What it does not prove
	pack/demo/architecture-review-demo	and reproduce committed goldens.	the selected tools and fixtures.

Local commands

Use these commands as orientation, not as proof that production is ready:

```
# Platform validation
cd platform
go test ./...
go test -race ./...
go vet ./...
go run ./cmd/aapctl validate

# Writes ../out/proofs/phase1-proof.json relative to platform/
go run ./cmd/aapctl prove
```

```
# MCP server validation
cd skills-pack/examples/microservices-system-design-mcp-server
make test
go test -race ./...
go vet ./...
make build
```

```
# MCP demo validation
cd skills-pack/demo/architecture-review-demo
export MCP_SERVER_BIN=../../examples/microservices-system-design-mcp-server/mcp-server
make demo
make validate
```

```
# Skills-pack index / documentation consistency after skill or agent changes
cd skills-pack
python3 validation/tools/skill_audit.py --emit-index
```

Release-gate expectations vs current automation

[docs/release-gate-thresholds.md](#) defines the intended BA Agent release gates. Current evidence should be read honestly:

Gate	Current status in repo	Notes
Manifest tests	Implemented/tested	Runtime tests and aapctl validate cover manifest loading and failure modes.
Tool-denial tests	Implemented; partly directly tested	Off-manifest, schema violation, blocked action, scope mismatch, pre-start denial, and approval boundary paths have direct tests. Missing-contract and contract-version denial paths exist in runtime code, while direct test coverage is [VERIFY] .

Gate	Current status in repo	Notes
Memory leakage tests	Implemented/tested	Runtime tests and proof report expect 0 cross-engagement records.
Benchmark evals	Partially evidenced	BA Agent <code>eval-results.json</code> has 6/6 results across 3 trials; release-gate thresholds call for no regression but broader production baselines remain <code>[VERIFY]</code> .
Prompt-injection tests	Partially evidenced	BA Agent eval case A-002 records injection refusal across 3 trials; platform-level prompt-injection suite is not separately shown.
Approval-gate accuracy	Partial	Runtime covers specific approval behavior; <code>docs/release-gate-thresholds.md</code> expects N >= 50 golden cases, which is not evidenced in the local harness.
Trace completeness	Partial	Harness emits trace-shaped records and audit events; docs explicitly say these are not live OTEL spans.
Memory citation enforcement	Implemented/tested for harness writes	Runtime rejects citation run mismatches and validates memory schema.
Audit coverage	Implemented for covered harness actions	Full production audit coverage for overrides, evals, releases, purges, and live operations remains broader than the local proof.
Telemetry payload mode	Implemented/tested	Active/platform-ready manifests must use <code>hash-and-reference</code> .

Boundaries, trade-offs, and failure modes

Decision or boundary	Benefit	Trade-off	Failure mode to watch	Validation path
Local deterministic harness before hosted runtime	Core contracts are testable without SDK/runtime uncertainty.	Hosted runtime integration remains unproven.	Treating proof harness success as production runtime readiness.	Keep <code>docs/runtime-verification-notes.md</code> open items visible; require hosted-runtime spike before production commitment.
Manifest and contract files in git	Reviewable, diffable, CI-validatable source of truth.	Slower than dynamic UI-driven changes.	Manifest/contract drift, stale version pins, or unreviewed source path changes.	Run <code>aapctl validate</code> , platform tests, and schema validation after any contract change.
Default-deny tool gateway	Prevents off-manifest and high-risk execution by construction.	Requires explicit contract work for every allowed action.	A tool appears in prose/agent instructions but lacks a contract or manifest pin.	Runtime should deny; add tests for any new tool path.

Decision or boundary	Benefit	Trade-off	Failure mode to watch	Validation path
Engagement-scoped memory	Limits cross-client or cross-engagement leakage.	Current harness memory is in-process and not a real vector/persistent store.	Assuming persistence, recall quality, or multi-tenant isolation beyond local semantics.	Add persistent-memory integration tests when memory backend is introduced.
Hash-and-reference telemetry posture	Reduces sensitive payload exposure in traces/audits.	Debugging requires payload lookup discipline and retention policy.	Raw payloads appear in active/platform-ready traces or logs.	Telemetry payload-mode test plus log/audit review before production.
Deterministic MCP architecture tools	Reproducible outputs and precise golden drift detection.	Encoded rules may miss context-specific architecture constraints.	Treating MCP output as accepted architecture.	Human architect review; update service tests and goldens only with rationale.
Golden fixture validation	Makes behavior drift explicit.	Goldens can ossify flawed behavior if refreshed casually.	make refresh hides a regression.	Require review note explaining why golden drift is intentional.
Skills as progressive-disclosure knowledge	Keeps AI sessions focused while retaining depth.	Skill prose can drift from runtime implementation.	Logo-list or stale guidance winning over code/tests.	Generated index, skill audit, and evidence precedence rules.

Known **[VERIFY]** / drift items for future sessions:

- Runtime SDK, managed hosting target, OTel GenAI, identity, and memory choices are open verification items in **docs/runtime-verification-notes.md**.
- **skills-pack/README.md** includes historical count language while **skills-pack/.claude/INDEX.md** and **Ranking.md** show the current 35-skill inventory. Use the generated index for inventory.
- Root **.claude/CLAUDE.md** mentions **.mcp.json** and **.claude/settings.json**, but those files are not present in this checkout. Do not assume root MCP bindings or Claude settings are active without verifying the current filesystem.
- BA Agent evaluation prose has stale lines, but **eval-results.json** provides concrete executed results. Prefer executable/generated evidence over prose summaries when they conflict.

How to work safely in this repo

Source-of-truth precedence

Use this order when evidence conflicts:

1. Code, schemas, CI, executable tests, generated proof/eval outputs.
2. Generated indexes and machine-readable release/eval artifacts.

3. Current status docs under [docs/](#), [governance/](#), and package indexes.
4. Planning prose, dated snapshots, diagrams, historical workspaces, or copied archives.

Canonical workspace rule: this repository wins over historical snapshots listed in [.claude/CLAUDE.md](#).

Update snapshots from here if needed; do not let snapshots overwrite this repo.

Change-impact map

Change type	Files likely affected	Required validation
Manifest field or status change	examples/ba-agent.manifest.yaml , schemas/agent-manifest.schema.json , runtime tests	<code>cd platform && go test ./... && go run ./cmd/aapctl validate</code>
New or changed AAP tool	tool-contracts/ , manifest allowed tools, runtime tests	Contract schema validation, input schema example validation, allowlist/version tests, audit payload tests
Runtime enforcement change	platform/internal/runtime/ *, docs/runtime-verification-notes.md , proof output	Platform tests, race tests, aapctl validate , aapctl prove , review proof diff
Blocked action or approval policy change	governance/aap-blocked-actions.yaml , guardrails, runtime tests	Blocked-action and approval-boundary tests
Memory behavior change	schemas/memory-record.schema.json , platform/internal/runtime/memory.go , runtime tests	Memory isolation, classification, PII, and citation tests
MCP server tool change	internal/services/ , internal/tools/ , contracts, testdata	MCP tests/race/vet/build; update goldens only when behavior change is intentional
Demo fixture or output change	skills-pack/demo/architecture-review-demo/input , out , golden	<code>make demo && make validate</code> ; document intentional drift
Skill or agent inventory change	skills-pack/.claude/skills/ , skills-pack/.claude/agents/ , skills-pack/.claude/INDEX.md , Ranking.md	<code>python3 validation/tools/skill_audit.py --emit-index</code> ; update inventory docs if counts changed
Instruction OS change	instruction-os/Persona/ , instruction-os/skills/ , validation history	Edit active Persona source first; update validation artifacts only after running the relevant stress/audit pass
Production-readiness claim	Runtime/deployment docs, monitoring, runbooks, authz, SLOs, rollback, ownership	Require deployment topology, authn/authz, observability, SLOs, alerting, runbooks, rollback, and owner evidence before using "production-ready"

AI coding-session rules

- Lead with repository evidence. Do not infer implementation from PRD language unless code/tests/config support it.
- Do not upgrade the local proof harness into a production runtime in prose.
- Do not add new tools to an agent instruction without a contract, manifest pin, audit shape, and tests.
- Do not claim current ecosystem facts unless the dated source was re-verified; otherwise mark **[VERIFY]**.
- If architecture decisions are open, route to the appropriate architect agent before documenting a decision.
- Treat generated outputs under **out/** and demo goldens as evidence. If they change, explain why.
- Avoid broad refactors across **skills-pack/**, **instruction-os/**, and **platform/** in one change unless the cross-layer contract is explicit.

Quick orientation paths

Need	Start here
Repository purpose and folder map	README.md
AI-session source-of-truth rules	.claude/CLAUDE.md
AAP product/runtime intent	governance/PRD_AaraMinds_Agent_Platform_v1.3.md
AAP guardrails and blocked actions	governance/aap-guardrails-checklist.md , governance/aap-blocked-actions.yaml
Local harness CLI	platform/cmd/aapctl/main.go
Runtime enforcement	platform/internal/runtime/engine.go , loader.go , memory.go , schemas.go , proof.go
Runtime tests	platform/internal/runtime/runtime_test.go
BA proof flow and runtime boundary	docs/ba-agent-proof-flow.md , docs/runtime-verification-notes.md
Release thresholds	docs/release-gate-thresholds.md
Manifest and contracts	examples/ba-agent.manifest.yaml , tool-contracts/*.contract.yaml , schemas/
Current proof output	out/proofs/phase1-proof.json
Skills and agents inventory	skills-pack/.claude/INDEX.md
BA Agent package	skills-pack/agent-packages/aara-business-analyst/
MCP server	skills-pack/examples/microservices-system-design-mcp-server/
MCP demo	skills-pack/demo/architecture-review-demo/

Need	Start here
Instruction OS active source	instruction-os/Persona/
Communication skills	instruction-os/skills/

Evidence anchors

This document avoids line-range citations so references do not rot after edits. All path-level evidence is valid as of commit [a926c5abb45c3024f0f30a0ae5ff4a931d2202e8](#).

Evidence path	Claim supported
README.md	Canonical workspace, AAP implementation home, top-level layout, quick proof command.
.claude/CLAUDE.md	Workspace source-of-truth rule, skills-pack vs Instruction OS split, canonical-vs-snapshot policy, AI-session quality bar.
.github/workflows/ci.yml	CI gates for platform, MCP server, and demo validation.
governance/PRD_AaraMinds_Agent_Platform_v1.3.md	AAP positioning, non-goals, runtime assumptions, proof definitions, data model intent, minimum artifacts.
governance/aap-guardrails-checklist.md	Baseline AAP guardrails and release blockers.
governance/aap-blocked-actions.yaml	Blocked action taxonomy and default boundaries.
docs/runtime-verification-notes.md	Local harness boundary, open runtime verification items, current automated coverage summary.
docs/ba-agent-proof-flow.md	BA Agent proof flow and current implementation coverage of steps 2-10.
docs/release-gate-thresholds.md	Release-gate thresholds and approval golden-suite expectation.
platform/go.mod	Platform module path, Go version, schema/YAML dependencies.

Evidence path	Claim supported
platform/cmd/aapctl/main.go	validate and prove CLI behavior.
platform/internal/runtime/*.go	Runtime types, loader, schema validation, engine, memory, proof generation.
platform/internal/runtime/runtime_test.go	Automated runtime coverage.
schemas/*.schema.json	AAP manifest, tool contract, audit event, memory record, eval run, approval request schemas.
examples/ba-agent.manifest.yaml	BA Agent manifest: skills, tools, memory, approval, telemetry, evaluation gate.
tool-contracts/*.contract.yaml	Tool contract pins, schemas, permissions, failure modes, audit payloads.
out/proofs/phase1-proof.json	Current generated phase-1 proof output.
skills-pack/.claude/INDEX.md	Generated inventory of skills, agents, hooks, and pattern cards.
skills-pack/examples/microservices-system-design-mcp-server/README.md	MCP server purpose, tool list, layering, tests, production boundary.
skills-pack/examples/microservices-system-design-mcp-server/cmd/server/main.go	MCP transport selection and stderr logging discipline.
skills-pack/examples/microservices-system-design-mcp-server/internal/mcpserver/server.go	Tool registration and service composition.
skills-pack/demo/architecture-review-demo/README.md, demo_runner.py, validate_outputs.py	Authentic MCP demo, stdio client behavior, golden comparison.
skills-pack/agent-packages/aara-business-analyst/*	BA Agent spec, eval results, release gate, monitoring, rollback, adapter contracts.
instruction-os/README.md, instruction-os/Persona/README.md	Instruction OS active source, load order, personas, validation model.